An Internship Report

on

Vison based tracking and navigation in Underwater Vehicles

Submitted in the fulfilment of

Summer Research Internship

Submitted by

Sargam Malik

Under the supervision of

Dr. Jagadeesh Kadiyam

CAIR@IIT Mandi

Assistant Professor

Centre for Artificial Intelligence and Robotics

Indian Institute Of Technology Mandi

ACKNOWLEDGEMENT

I am deeply grateful to all those who contributed to making my summer research internship a rewarding and enriching experience. First and foremost, I would like to extend my sincerest thanks to **Dr. Jagadeesh Kadiyam** for his invaluable guidance, unwavering support, and constant encouragement throughout the duration of my internship. His mentorship provided me with the opportunity to learn, grow, and develop a deeper understanding of my field of study, for which I am profoundly thankful.

I would also like to express my appreciation to the entire team at **CAIR, IIT Mandi** for their assistance and collaboration. Their openness in sharing knowledge and providing constructive feedback was instrumental in the successful development of my project.

A special note of gratitude goes to **Mr. Ritulesh Mohan**, whose continuous support made my time at the institution both productive and enjoyable.

To everyone who played a part in my academic journey during this internship, thank you for being an integral part of my personal and professional growth.

Date: 20/07/2024

Place: IIT Mandi, Mandi, Himachal Pradesh - 175005

TABLE OF CONTENTS

ABSTRACT

This internship project focuses on developing and implementing a camera-based navigation and tracking system for underwater vehicles. The primary objective was to enhance the accuracy and reliability of underwater navigation and object tracking using visual data, which is critical for various applications such as marine research, underwater inspection, and search and rescue operations. The system integrates advanced image processing techniques and computer vision algorithms to interpret and utilize real-time video feeds from onboard cameras. Key components of the project included the calibration of the camera setup, development of image processing algorithms to detect and track underwater landmarks and objects, and the integration of these algorithms with the vehicle's navigation system. This project contributes to the field of underwater robotics by providing a robust solution for visual navigation and tracking, with potential applications in autonomous underwater vehicles (AUVs) and remotely operated vehicles (ROVs).

INTRODUCTION

The rapid advancement of underwater exploration technologies has significantly enhanced our understanding of the vast, uncharted territories beneath the ocean's surface. In this context, our project focuses on developing a robust and efficient camera-based navigation system for underwater vehicles, leveraging state-of-the-art techniques in object detection, object tracking, and visual odometry. This report provides a comprehensive overview of our project's objectives, methodologies, and outcomes.

The primary goal of this project is to enable autonomous underwater vehicles (AUVs) to navigate complex underwater environments accurately and efficiently. Traditional navigation systems often rely on sonar and other acoustic sensors, which, while effective, have limitations in terms of resolution and range. Camera-based navigation offers a complementary approach, providing high-resolution visual data that can be crucial for detailed mapping and obstacle avoidance.

Object Tracking: Building on the object detection module, our object tracking system ensures continuous monitoring of identified objects as the AUV moves through its environment. This continuous tracking is vital for maintaining situational awareness and for making informed navigational decisions.

Visual Odometry: To accurately estimate the vehicle's position and orientation over time, we have implemented a visual odometry system. By analysing the sequence of images captured by the camera, the system calculates the motion of the AUV, enabling precise navigation even in the absence of external positioning systems like GPS, which are ineffective underwater.

LITERATURE REVIEW

Recent advancements in visual-based navigation for autonomous underwater vehicles (AUVs) have highlighted the potential of camera-based systems to improve underwater navigation and inspection. The study "**Visual-Based Navigation Strategy for Autonomous Underwater Vehicles in Monitoring Scenarios**" explores real-time image processing to enhance situational awareness and navigation accuracy in dynamic environments. The efficacy of RGB-D cameras in underwater inspection tasks is demonstrated in "**RGB-D Camera Navigation for Autonomous Underwater Inspection Using Low-Cost Micro AUVs**," showcasing the integration of depth information with visual data for precise navigation. Addressing the challenges of underwater imaging, "**Color Correction Between Gray World and White Patch**" presents methods to correct color distortions, crucial for reliable visual data. Additionally, "**Implementation of Visual Odometry for Underwater Robot on ROS by Raspberry Pi 2**" details the application of visual odometry using a low-cost platform, enhancing position estimation capabilities. Finally, "**Acoustic-Optic Assisted Multisensor Navigation for Autonomous Underwater Vehicles**" discusses the fusion of acoustic and optical sensors, providing a robust framework for navigation in complex underwater environments. Together, these studies form a comprehensive foundation for developing advanced, reliable camera-based navigation systems for AUVs.

3.1.1 **Visual-Based Navigation Strategy for Autonomous Underwater Vehicles in Monitoring Scenarios**

This work proposes a navigation strategy for Autonomous Underwater Vehicles (AUVs) using a single bottom-looking camera and altitude information for linear velocity estimation. This reduces the need for multiple sensors by utilizing the existing payload for both monitoring and navigation. The method employs a monocular Visual Odometry (VO) technique, switching between homography and epipolar models to resolve motion and scale ambiguity using altitude data. An Extended Kalman Filter (EKF) combines visual-based linear velocity with attitude and depth measurements for trajectory estimation. The strategy was tested on real data from the Zeno AUV, showing a maximum absolute error of 2.16m compared to DVL-based dead-reckoning. The VO algorithm includes preprocessing for image correction, feature detection using SURF, motion estimation with model switching validated by chirality checks, and scale factor computation from altitude readings. The navigation algorithm integrates these linear velocity estimates with attitude and depth data in an EKF to refine velocity updates and estimate the AUV's trajectory.

- **Navigation Strategy**:
  - Utilizes a single bottom-looking camera and altitude information.
  - Reduces the need for multiple sensors by using existing payload for both monitoring and navigation.
- **Monocular Visual Odometry (VO)**:
  - Switches between homography and epipolar models for motion estimation.
  - Resolves scale ambiguity using altitude data.
  - Compares estimated linear velocities with Doppler Velocity Log (DVL) readings.
- **Extended Kalman Filter (EKF)**:
  - Combines visual-based linear velocity with attitude and depth measurements.
  - Refines velocity updates and estimates the AUV's trajectory.
- **Algorithm Workflow**:
  - **Preprocessing**: Corrects images for refraction and lens distortion, and enhances contrast using CLAHE.
  - **Feature Detection**: Uses SURF algorithm for robust feature extraction and matching.

    - **Motion Estimation**: Adapts geometric models based on scene structure and validates them with cheirality checks.
    - **Scale Factor Computation**: Determines scale factor by comparing altitude readings with triangulated 3D points.
- **Performance Assessment**:
    - Tested on real data from the Zeno AUV.
    - Maximum absolute error of 2.16m compared to DVL-based dead-reckoning.

3.1.2 RGB-D Camera-based Navigation for Autonomous Underwater Inspection using Low-cost Micro AUVs

This paper explores the use of commercial RGB-D cameras for autonomous underwater structure inspections by micro autonomous underwater vehicles (µAUVs). The focus is on close-distance navigation around structures with unknown geometries. The proposed method involves a combination of global and local path planning, utilizing real-time data from an Intel RealSense D435i RGB-D camera for local shape tracking and adaptive navigation. The approach aims to improve inspection efficiency and data quality. The effectiveness of this method is demonstrated through simulations and experiments. All software and hardware designs used in this study will be made available online.

Global Mission Planning

- **Inspection Path**: Generated to ensure coverage of the target area (e.g., helix path around a cylindrical fish net pen).
- **Absolute Positioning**: Relies on an absolute localization system for global path planning and docking facility approach/departure.
- **Mission Execution**: The global planner provides attitude setpoints and fuses them into the local path planning module.

Underwater Structure Detection

1. **3D Point Cloud**: The sidewards-facing RGB-D camera provides a 3D point cloud of the structure.
2. **Camera Calibration**: Compensates for refraction effects in water and acrylic glass, requiring advanced calibration methods.

3. **Filtering and Clustering**:
    - **Statistical Outlier Removal**: Filters noise and outliers using a k-clustering algorithm.
    - **Euclidean Clustering**: Identifies the largest and best-connected structure within the point cloud.
4. **Structure Tracking**: Approximates the structure with linear segments tracked over time using an Extended Kalman Filter (EKF).

RGB-D Camera Calibration

- **Challenges**: Infrared light attenuation underwater and camera housing increase complexity.
- **Compensation**: Calibration to account for refractive indexes of water, acrylic, and air.

Local Path Planning and Attitude Control

- **Local Path Generation**: Follows the structure shape at a desired distance using EKF-tracked linear segments.
- **Attitude Control**:
    - **Thrust Scaling**: Prioritizes attitude control commands over forward thrust to avoid thruster saturation.

Hardware Platform

- **HippoCampus µ\muµAUV**:
    - 50 cm length, mostly 3D printed.
    - Equipped with Intel RealSense D435i RGB-D camera and two RaspberryPi 4 companion computers.
    - Uses a PixRacer flight computer for low-level control within the PX4-firmware stack.
    - Camera operates at 15 fps due to computational limitations.

Results and Observations

- **Detection Example**: Curved surface detection shown as a point cloud with front/top view using the D435i camera.

3.1.3. Implementation of Visual Odometry Estimation for Underwater Robot on ROS by using RaspberryPi 2

This research focuses on implementing a visual odometry estimation algorithm on a RaspberryPi 2 for an autonomous underwater vehicle (AUV). The system utilizes the Robot Operating System (ROS) for managing inter-process communications. The visual odometry estimation leverages image processing techniques to estimate the AUV's movement and orientation changes using frames captured by a high-definition camera attached to the AUV. The visual odometry is integrated with an Inertial Measurement Unit (IMU) and a depth sensor to improve accuracy. The IMU helps determine correct answers for the homography motion equation, while the pressure sensor resolves image scale ambiguity.

The odometry estimation involves calculating the robot's movement by comparing consecutive image frames using the Shi-Tomasi method and Lucas-Kanade optical flow, with outliers removed by RANSAC. Covariance matrix estimation is done to assess system uncertainty using forward and backward error propagation, Jacobian, and Singular Value Decomposition methods.

The RaspberryPi 2, chosen for its cost-effectiveness and computational power, runs Ubuntu 14.04 with ROS Indigo for managing the AUV's software components. The underwater robot, equipped with eight thrusters for 6-degree of freedom movement, an IMU for orientation, and a pressure sensor for depth measurement, uses a bottom-mounted camera to capture floor images for odometry estimation.

3.1.4. Acoustic-optic assisted multi sensor navigation for autonomous underwater vehicles

Autonomous Underwater Vehicles (AUVs) rely on various sensors for navigation due to the challenges posed by the underwater environment, where electromagnetic waves attenuate rapidly. Traditional methods like Doppler Velocity Log (DVL) and acoustic transponders are commonly used, but they have limitations. Visual Odometry (VO) and Acoustic Odometry (AO) are emerging as complementary techniques, though each has its challenges. Combining multiple sensors can enhance navigation accuracy but increases computational demands.

Challenges and Current Methods

- Electromagnetic Wave Attenuation: Prevents the use of GPS underwater.
- Dead Reckoning (DR): Uses DVL for velocity information but is affected by environmental interferences.
- Transponder-Based: Requires fixed baselines, which are costly to deploy and maintain.
- Geophysical Data-Based: Utilizes optical and acoustic data, which need external sensors.

Sensor Fusion

To improve navigation accuracy and robustness, a sensor fusion framework based on a parallel federated Unscented Kalman Filter (UKF) is proposed. This framework combines VO and AO methods to leverage their complementary strengths and uses the trapezoid formula to enhance accuracy while reducing computational burden.

3.1.5 Color correction between Gray World and White Patch

This publication, "Color correction between Gray World and White Patch," discusses the development of a new chromatic correction algorithm called Automatic Color Equalization (ACE). The authors, Alessandro Rizzi, Carlo Gatta, and Daniele Marini from the University of Milano, aim to merge two existing approaches to color correction—Gray World and White Patch—to create a more effective method for achieving color constancy in digital images.

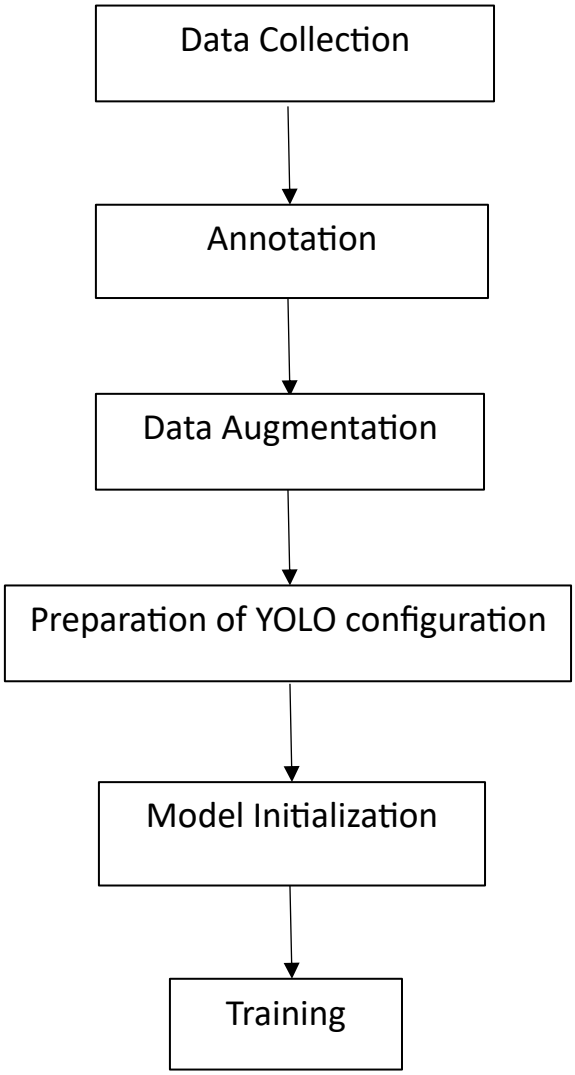
Key Concepts and Mechanisms

- **Gray World (GW)**: Assumes the average color in a scene should be neutral gray. It centers the histogram dynamically, similar to camera exposure control.
- **White Patch (WP)**: Identifies the lightest patch in an image as a reference for white, mimicking the human visual system's color constancy mechanism.
- **Retinex Algorithm**: Belongs to the WP family, using a reset mechanism to identify white reference areas in an image.
- **Automatic Color Equalization (ACE)**: Combines GW and WP approaches, performing color constancy even when based on the Gray World method. ACE maintains the Retinex principle that color sensation results from comparing spectral lightness values across the image.

- **Algorithm Details**

1. **Chromatic/Spatial Adaptation**:
    - Each pixel's output value is computed based on the differences in intensity between it and neighbouring pixels, weighted by distance and relative lightness.
    - **Distance Function (d(·))**: Balances local and global adaptation effects.
    - **Relative Lightness Appearance Function (r(·))**: Enhances contrast for WP behavior and centers dynamics for GW behavior.
    - **Normalization**: Compensates for edge effects by normalizing contributions from surrounding pixels.
2. **Dynamic Tone Reproduction Scaling**:
    - **Linear Scaling**: Adjusts the dynamic range of the output image to fit standard display ranges.
    - **WP/GW Scaling**: Uses white and gray references to further adapt the dynamic range, ensuring balanced tone reproduction.

OBJECT DETECTION IN LIVE VIDEO



Training and Detection Process:

1. **Data Collection**: Gather a diverse dataset of underwater pipe images.
2. **Annotation**: Annotate the images with bounding boxes around the pipes.
3. **Data Augmentation**: Augment the dataset by applying transformations such as rotation, flipping, and adjusting brightness to increase diversity.
4. **Preparation of YOLO Configuration**: Configure YOLOv8 parameters .
5. **Model Initialization**: Initialize the YOLOv8 model with pre-trained weights on a large dataset like COCO
6. **Training**: Train the YOLOv8 model on the annotated dataset using a large GPU cluster. This involves optimizing the model's parameters to minimize the detection error.

   >> **yolo train data=coco8.yaml model=yolov8n.pt epochs=10 lr0=0.01**
7. **Detection**: Save and use the trained model in detection script.

Key Principles

1. **Model Loading**:
   - The YOLO (You Only Look Once) model is a state-of-the-art object detection model that can detect objects in images and videos in real-time. In this code, a pre-trained YOLOv8 model is loaded using the Ultralytics library.

2. **Webcam Initialization**:
   - The webcam is initialized using OpenCV's cv2.VideoCapture(0), where 0 denotes the default camera. The frame size is optionally set to 640x480 pixels.

3. **Real-time Frame Capture and Processing**:
   - The code enters an infinite loop to continuously capture frames from the webcam.
   - Each captured frame is passed to the YOLO model for object detection.

4. **Object Detection**:
   - The YOLO model processes each frame and returns the detection results, which include bounding boxes, class labels, and confidence scores for detected objects.

- Bounding boxes and labels are drawn on the frame using OpenCV functions.

5. **Saving Detected Objects**:
   - For each detected object, a sub-image corresponding to the bounding box is extracted from the frame.
   - The extracted sub-image is saved to a specified directory with the object label and confidence score overlayed.

6. **Displaying the Frame**:
   - The processed frame with bounding boxes and labels is displayed in a window named 'YOLOv8 Detection'.

7. **Resource Management**:
   - The loop continues until the 'q' key is pressed, at which point the webcam is released and the OpenCV windows are closed to free up resources.

OBJECT TRACKING

The object tracker in this code works primarily based on the following principles:

1. **Object Detection and Tracking**:
    - YOLOv8 Model: The YOLO (You Only Look Once) model is used for real-time object detection and tracking. YOLOv8 is a state-of-the-art model that detects objects within an image or video frame and assigns unique track IDs to each detected object, allowing the tracking of objects across multiple frames.
    - Tracking Persistence: The model maintains persistent tracks across frames using track IDs, enabling the system to follow the same object even as it moves within the frame.

2. **Data Structures for Tracking**:
    - Track History: A defaultdict is used to store the history of tracked objects. Each key in the dictionary corresponds to a track ID, and the value is a list of coordinates (x, y) representing the object's center in successive frames.
    - Track Timestamps: Another defaultdict is used to store timestamps of the last detection for each track ID. This helps in identifying and removing stale tracks that have not been updated for a certain period.

3. **Servo Control for Object Following:**
    - Remapping Coordinates: The x-coordinate of the target object is remapped to a servo angle using a linear transformation. This mapping ensures that the object's position within the camera frame is translated to an appropriate angle for the servo motor.
    - Servo Angle Smoothing: An exponential smoothing technique is applied to the calculated servo angle to ensure smooth movements of the servo motor, avoiding sudden jumps or jitters.

4. **User Interaction:**

    - Manual Target Selection: The user can manually input the ID of the target object to follow by pressing a key ('p'). This allows for interactive selection and tracking of specific objects.

Code Overview

1. **Import Libraries:**

    - Essential libraries such as cv2, NumPy, serial, and time are imported along with defaultdict from collections.

2. **Model and Video Initialization:**

    - The YOLOv8 model is loaded using the specified model path.
    - Video capture is initialized using cv2.VideoCapture(1).
    - Data structures for tracking history and timestamps are initialized.

3. **Servo Control Setup:**

    - Constants for mapping camera coordinates to servo angles are defined.
    - Initial servo angle and smoothing factor are set.
    - Serial communication with an Arduino is established to control the servo motor.

4. **Main Loop for Real-time Detection and Tracking:**

    - The code enters a loop where each frame is read from the video feed.
    - The frame is processed by the YOLOv8 model to detect and track objects.
    - Track history and timestamps are updated for each detected object.
    - Stale tracks (not updated for 60 seconds) are removed.
    - Tracking information is visualized on the frame, and tracking lines are drawn.

- If the user has selected a target object, the servo angle is calculated based on the target object's position and sent to the Arduino.
- The frame is displayed, and the loop continues until the 'q' key is pressed.
- The user can also press 'p' to manually input a target object ID.

5. **Resource Cleanup:**

   - Upon exiting the loop, video capture and serial communication are properly closed

CAMERA CALLIBERATION

Camera calibration is the process of estimating the parameters of a camera lens and image sensor. This process allows us to accurately understand the relationship between the 3D real world and the 2D images that the camera captures. The main objective is to find the intrinsic and extrinsic parameters of the camera.

Key Concepts in Camera Calibration

1. **Intrinsic Parameters**: These are the parameters that describe the internal characteristics of the camera. They include:
    - **Focal length (fx, fy)**: These determine the scale of the image in the x and y dimensions.
    - **Principal point (cx, cy)**: This is the point where the optical axis intersects the image plane.
    - **Skew coefficient**: This defines the skewness between the x and y axis of the sensor.
    - **Distortion coefficients**: These account for lens distortion, including radial distortion (barrel or pincushion effects) and tangential distortion (due to lens misalignment).

2. **Extrinsic Parameters**: These parameters describe the position and orientation of the camera in the world. They include:
    - **Rotation vector (rvecs)**: This represents the rotation of the camera relative to the world coordinate system.
    - **Translation vector (tvecs)**: This represents the translation of the camera relative to the world coordinate system.

3. **Distortion Coefficients**: These coefficients describe the distortion introduced by the camera lens. The main types are:

- **Radial distortion**: This causes straight lines to appear curved. Common types include barrel distortion (lines curve outward) and pincushion distortion (lines curve inward).
- **Tangential distortion**: This occurs when the lens and the image plane are not perfectly parallel.

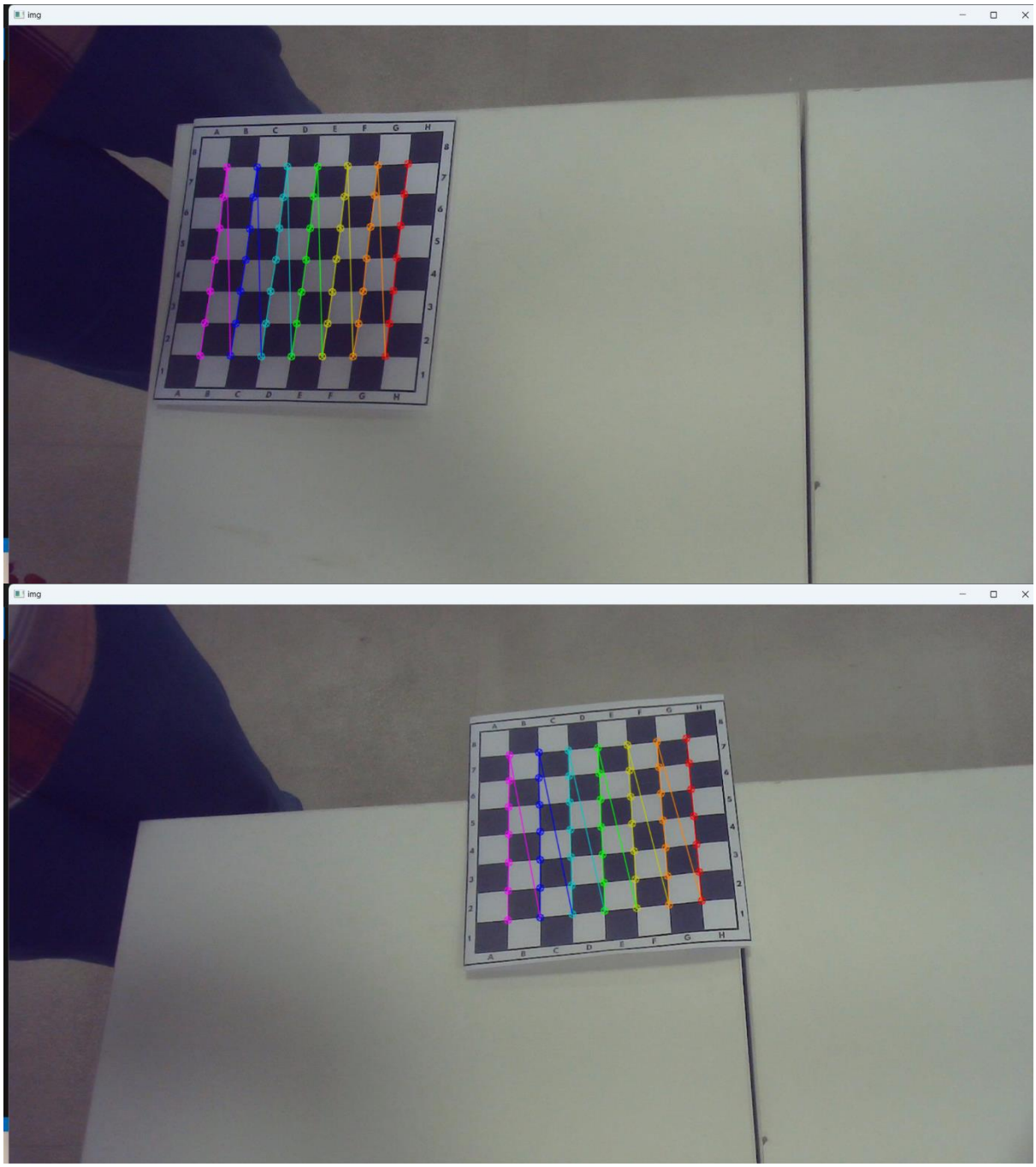
Principles of Camera Calibration

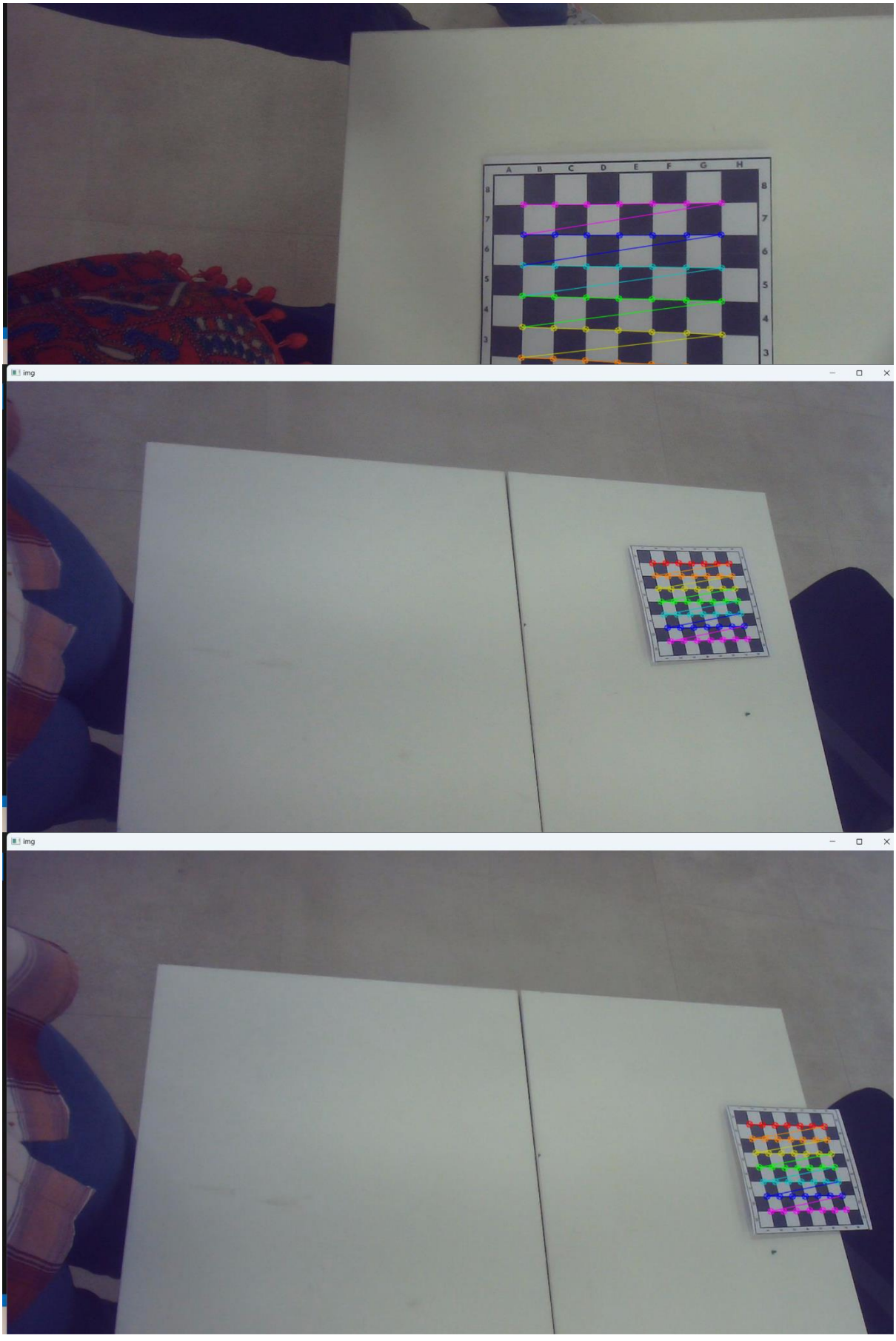
1. **Image Formation**: The camera captures a 3D scene and projects it onto a 2D image plane. This projection involves a transformation from the 3D world coordinates to the 2D image coordinates, which is influenced by both intrinsic and extrinsic parameters.
2. **Chessboard Pattern**: A common method for camera calibration involves using a known pattern, such as a chessboard. The corners of the chessboard provide a set of known 3D points and their corresponding 2D image points.
3. **Finding Corners**: The algorithm detects the corners of the chessboard in the image. These corners are used to establish a correspondence between the 3D world coordinates and the 2D image coordinates.
4. **Optimization**: Using these correspondences, the camera calibration algorithm solves a set of equations to minimize the reprojection error. The reprojection error is the difference between the observed image points and the projected 3D points using the estimated camera parameters.
5. **Reprojection Error Calculation**: This error provides a measure of how well the estimated parameters fit the observed data. It is computed by projecting the known 3D points back onto the image plane using the estimated parameters and comparing these projections with the observed 2D points.

Steps in Camera Calibration Using the Provided Code

1. **Prepare Object Points**: Define the 3D coordinates of the chessboard corners in real-world space. These are uniformly spaced points on the chessboard.
2. **Capture Image Points**: Detect the 2D coordinates of the chessboard corners in each calibration image.

3. **Calibrate the Camera**: Use the object points and image points to estimate the camera's intrinsic and extrinsic parameters. This involves solving for the camera matrix (intrinsic parameters) and distortion coefficients.
4. **Undistort Images**: Use the obtained parameters to correct for distortion in the captured images, resulting in more accurate and visually pleasing images.
5. **Evaluate the Calibration**: Compute the reprojection error to assess the accuracy of the calibration.




A B C D E F G H
8 7 6 5 4 3
img
img

VISUAL ODOMETRY USING MONOCULAR CAMERA

Visual odometry is the process of determining the location and orientation of a camera by analyzing a sequence of images. Visual odometry is used in a variety of applications, such as mobile robots, self-driving cars, and unmanned aerial vehicles. This example shows you how to estimate the trajectory of a single calibrated camera from a sequence of images.

Implementation of visual odometry

Imports and Initial Setup:

- Import necessary libraries: os, NumPy, cv2, sympy, tqdm, time, matplotlib.pyplot, pytransform3d, cycler, mpl_toolkits.mplot3d.

Define CameraPoses Class:

- **Initialization (__init__):**
  - Set intrinsic parameters, create ORB detector, FLANN matcher, initialize world points, and current pose.
- **Static Method _load_images:**
  - Load and resize images from the specified directory.
- **Static Method _form_transf:**
  - Create a 4x4 transformation matrix from rotation and translation.
- **Method get_world_points:**
  - Return the array of world points.
- **Method get_matches:**
  - Detect and compute ORB features, match descriptors using FLANN, and return good matches.
- **Method get_pose:**
  - Compute the essential matrix, decompose it into rotation and translation, and return the transformation matrix.
- **Method decomp_essential_mat:**
  - Decompose the essential matrix, triangulate points, and return the best rotation and translation pair based on positive z-coordinates.

- **Method decomp_essential_mat_old:**
    - Older method to decompose essential matrix, triangulate points, and determine the correct transformation based on positive z-coordinates and relative scale.

Load Intrinsic Parameters:

- Load intrinsic parameters from a NumPy file.

Initialize Variables:

- Define skip_frames, data_dir, and create CameraPoses instance.
- Initialize lists for estimated path, camera poses, and start pose.

Capture Video:

- Initialize video capture from the webcam.
- Check if the camera opened successfully.

Process Frames in a Loop:

- Capture frames from the webcam in a loop.
- Compute matches and pose for each frame.
- Update the current pose and append it to the list of camera poses.
- Display FPS and pose information on the frame.
- Show the frame with annotations.

Release Video Capture:

- Release the video capture object.

Plot Results:

- Plot camera poses in 3D using pytransform3d.
- Plot the estimated path in 2D.
- Plot the 3D camera trajectory using matplotlib.

ORB and FLANN Algorithms Used in the Code

The code utilizes ORB (Oriented FAST and Rotated BRIEF) and FLANN (Fast Library for Approximate Nearest Neighbours) for feature detection, description, and matching.

ORB (Oriented FAST and Rotated BRIEF)

ORB is a robust, efficient feature detection and description algorithm that combines the FAST key point detector and the BRIEF descriptor with several enhancements to improve performance, especially in terms of rotation invariance and computational efficiency.

1. **FAST Keypoint Detector**:
    - FAST (Features from Accelerated Segment Test) is used to detect corners in the image. It works by considering a circle of pixels around each candidate pixel and checks for a segment of contiguous pixels that are either all brighter or all darker than the candidate pixel by a certain threshold.
    - FAST is computationally efficient but not scale or rotation-invariant.
2. **Orientation Assignment**:
    - ORB addresses the lack of rotation invariance in FAST by computing the orientation of the keypoints using the intensity centroid method. The orientation is the direction of the vector from the keypoint to the centroid of the patch around it.
3. **BRIEF Descriptor**:
    - BRIEF (Binary Robust Independent Elementary Features) generates a binary string descriptor for each keypoint by comparing the intensities of pairs of pixels within a patch around the keypoint.
    - BRIEF is efficient but not rotation invariant.
4. **Rotated BRIEF**:
    - ORB applies the orientation found by the intensity centroid method to rotate the BRIEF descriptors, making them rotation invariant.

5. **Scale-Invariant Pyramid**:
    - ORB uses an image pyramid to detect keypoints at multiple scales, ensuring scale invariance.

ORB provides a good balance between computational efficiency and robustness, making it suitable for real-time applications like visual odometry.

FLANN (Fast Library for Approximate Nearest Neighbors)

FLANN is a library for performing fast approximate nearest neighbor searches in high-dimensional spaces. It is particularly useful for matching feature descriptors between images.

1. **Indexing Structures**:
    - FLANN uses various tree-based and hashing-based indexing structures to speed up the nearest neighbor search process. In this code, the LSH (Locality-Sensitive Hashing) algorithm is used.
    - LSH hashes similar feature descriptors into the same bucket with high probability, allowing for efficient approximate nearest neighbor search.

2. **Parameters**:
    - table_number: Number of hash tables used.
    - key_size: Size of the hash key in bits.
    - multi_probe_level: Number of nearby buckets to check for matches.

3. **KNN Matching**:
    - The code uses knnMatch to find the k-nearest neighbors (k=2 in this case) for each descriptor from the first image in the second image.
    - A ratio test is applied to filter out poor matches. Only matches where the distance ratio between the nearest neighbor and the second nearest neighbor is below a threshold (0.5 in this case) are retained.

Integration in Visual Odometry

The combination of ORB and FLANN in the visual odometry pipeline follows these steps:

1. **Feature Detection and Description**:
    - ORB detects keypoints and computes descriptors in consecutive frames of the video stream.

2. **Feature Matching**:
    - FLANN finds matches between the descriptors of keypoints in consecutive frames.

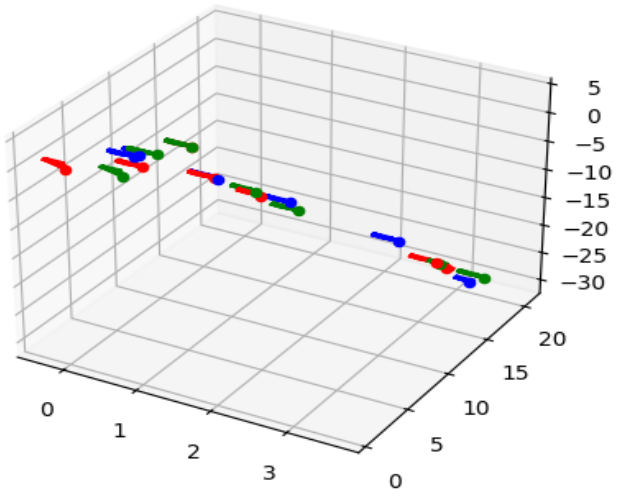
3. **Essential Matrix Calculation and Pose Estimation**:
    - The matched keypoints are used to compute the essential matrix, which encapsulates the rotation and translation between the two views.
    - The essential matrix is decomposed to extract the rotation (R) and translation (t) matrices.
    - These matrices are used to estimate the camera's pose relative to its position in the previous frame.

4. **Trajectory and 3D Reconstruction**:
    - The estimated poses are accumulated to build a trajectory of the camera.
    - Triangulation is performed to reconstruct the 3D positions of keypoints, contributing to a sparse 3D map of the environment.

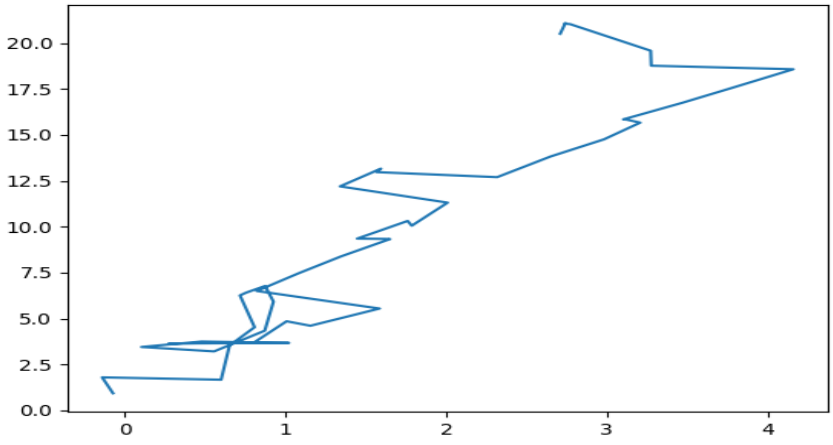
In summary, ORB provides an efficient way to detect and describe features in each frame, while FLANN accelerates the process of finding corresponding features between frames. Together, they enable the real-time estimation of camera motion and 3D scene structure in the visual odometry pipeline.

Implementation of the code:

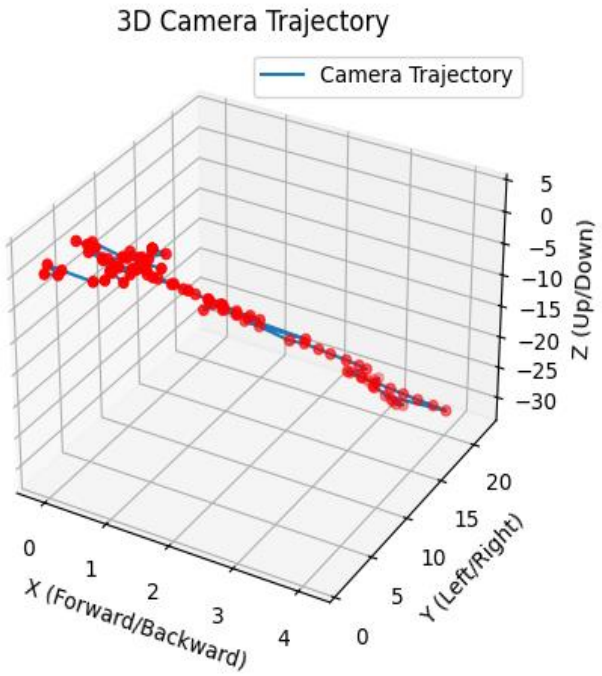
[Link to the implementation video]



a. This 3D plot shows the position of the camera in space at different keyframes. Different colored markers (r, g, b) represent the camera at different points in time.



b. A 2D projection of the camera's estimated trajectory on the XY plane (forward/backward and left/right directions). The jagged line represents the path that the camera has moved along.



c. This is a 3D plot of the full camera trajectory, labelled on the axes as X (Forward/Backward), Y (Left/Right), and Z (Up/Down). The red dots represent the camera's positions at each frame, while the blue line traces the path the camera followed.

GitHub Link:

[GitHub link for all the codes mentioned in the report]

FUTURE SCOPE

Adapting the provided algorithms and code for underwater vehicles presents unique challenges and opportunities. Here are some future directions and considerations for improving and expanding the use of visual odometry in underwater environments:

Enhanced Feature Detection and Matching:

Robust Feature Detectors: Develop or adapt feature detection algorithms that are robust to low visibility and distortion. Techniques like SIFT (Scale-Invariant Feature Transform) or SURF (Speeded-Up Robust Features) might offer better performance in challenging conditions.

Multimodal Sensors: Integrate data from other sensors, such as sonar or LiDAR, to complement visual information and improve feature detection and matching.

Adaptive Filtering and Noise Reduction

Advanced Filtering Techniques: Implement more sophisticated filtering techniques to reduce noise and improve the accuracy of feature tracking and pose estimation. Techniques like Kalman filters or particle filters could enhance robustness.

Adaptive Algorithms: Develop algorithms that adapt to varying visibility conditions and automatically adjust parameters for feature detection and matching.

Improved Pose Estimation and 3D Reconstruction

Improved Pose Estimation: Use advanced pose estimation techniques that account for complex underwater dynamics and environmental changes.

Dense 3D Reconstruction: Develop methods for dense 3D reconstruction that can create detailed maps of underwater environments, which can be useful for navigation and exploration.

Real-Time Processing and Computational Efficiency

Optimized Algorithms: Develop optimized algorithms that reduce computational complexity and improve real-time performance. Techniques like parallel processing or hardware acceleration (e.g., using GPUs) can help.

Efficient Data Handling: Implement efficient data handling and processing strategies to ensure that the system can operate within the constraints of the underwater vehicle's hardware.

Integration with Other Systems

Sensor Fusion: Integrate visual odometry with other navigation sensors, such as inertial measurement units (IMUs) and depth sensors, to improve overall accuracy and robustness.

Autonomous Navigation: Develop autonomous navigation systems that use visual odometry in conjunction with other data sources to enable advanced capabilities like path planning, obstacle avoidance, and automated exploration.

Handling Dynamic Environments

Dynamic Object Detection: Incorporate dynamic object detection and tracking to handle moving elements in the environment and reduce their impact on pose estimation.

Adaptive Algorithms: Develop adaptive algorithms that can adjust to changes in the environment and maintain accurate pose estimation despite dynamic conditions.

Long-Term Deployment and Robustness

Long-Term Calibration: Implement mechanisms for long-term calibration and drift correction to ensure consistent accuracy over extended missions.

Fault Detection and Recovery: Develop fault detection and recovery mechanisms to handle potential issues with the visual odometry system and maintain reliable operation.

CONCLUSION

During the internship, I gained significant experience in developing and optimizing a computer vision system for underwater vehicles, which involved object detection using YOLO, object tracking, camera calibration, and visual odometry with a monocular camera. Implementing YOLO for real-time object detection and integrating it with advanced tracking algorithms demonstrated the system's ability to efficiently track objects despite the challenges posed by underwater conditions such as low visibility and distortion. The camera calibration process was crucial for correcting lens distortions and obtaining accurate intrinsic and extrinsic parameters, which were essential for ensuring precise measurements and high-quality visual data. Developing visual odometry techniques with a monocular camera highlighted the complexities of depth estimation and motion tracking in a single-camera setup, but leveraging feature detection, matching algorithms, and essential matrix decomposition allowed for successful camera motion and trajectory estimation. The project underscored the importance of addressing environmental challenges through careful algorithm selection, tuning, and preprocessing, while balancing the need for real-time performance with computational efficiency. The integration of these components provided valuable insights into the intricacies of underwater vision systems and emphasized the need for continued innovation, including enhanced detection and tracking capabilities, sensor fusion, and algorithm optimization. Overall, the internship offered a comprehensive foundation in computer vision and robotics, equipping me with the skills and knowledge necessary for advancing future projects in underwater navigation and exploration.